复习课

关于考试

- 总成绩
  - 平时成绩 40%+考试 60%
- 考试题型
  - 简答题
  - 演算题
  - 推导题
- 考试要求
  - 时间：120分钟，1月19日下午
  - 闭卷考试：不能携带任何资料
  - 英文作答
  - 不带计算器：
    计算不复杂，根据题目需要给出分数形式结果即可

Performance Measure

Error rate and **accuracy** are the most commonly used performance measures in classification problems：

- Error rate is the proportion of misclassified samples to all samples
- Accuracy is the proportion of correctly classified samples instead

Error rate

$$E(f;D)=\frac{1}{m}\sum_{i=1}^{m}\mathbb{I}\left(f\left(\boldsymbol{x}_i\right)\neq y_i\right)$$

Accuracy

$$\begin{aligned}\operatorname{acc}(f;D) &= \frac{1}{m}\sum_{i=1}^{m}\mathbb{I}\left(f\left(\boldsymbol{x}_i\right)=y_i\right)\\ &= 1-E(f;D)\ .\end{aligned}$$

Performance Measure

- We often want to know "*What percentage of the retrieved information is of interest to users?*" and "*How much of the information the user interested in is retrieved?*" in applications like information retrieval and web search. For such questions, **precision** and **recall** are better choices.
- In binary classification, there are four combinations of the ground-truth class and the predicted class, namely *true positive*, *false positive*, *true negative*, and *false negative*. The four combinations can be displayed in a confusion matrix.

The confusion matrix of binary classification

Ground-truth class	Predicted class	
	Positive	Negative
Positive	TP	FN
Negative	FP	TN

Precision $P = \frac{TP}{TP + FP}$

Recall $R = \frac{TP}{TP + FN}$

Example: "Sheep" vs. "Goat" (Cont.)

Generalization

[泛化能力/推广能力]

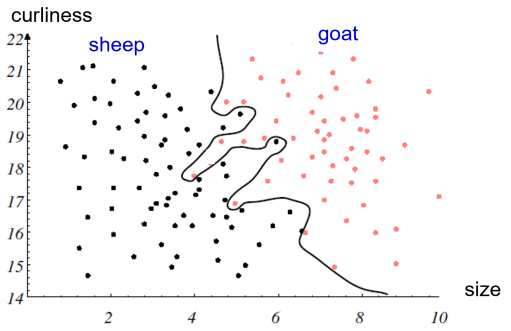
The ultimate goal!

The central aim of designing a classifier is to make correct decisions when presented with *novel* (*unseen*/*test*) patterns, not on training patterns whose labels are already known

Performance on the training set

Simplicity of the classifier

→ Tradeoff



Bias and Variance

- Bias measures the difference between the learning algorithm's expected prediction and the ground-truth label; that is, expressing the fitting ability of the learning algorithm;
- Variance measures the change of learning performance caused by changes to the equal-sized training set, that is, expressing the impact of data disturbance on the learning outcome;

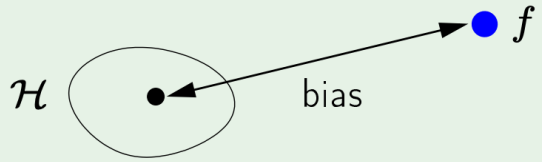
The bias-variance decomposition tells us that the generalization performance is jointly determined by the learning algorithm's ability, data sufficiency, and the inherent difficulty of the learning problem.

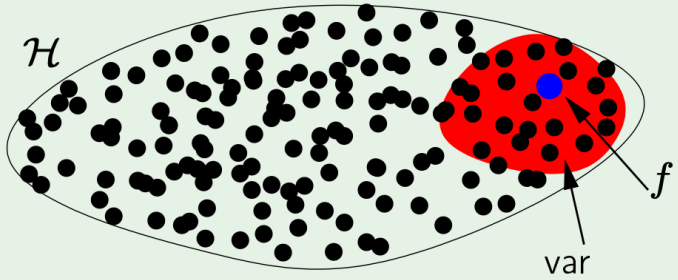
$$\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

In order to achieve excellent generalization performance, a small bias is needed by adequately fitting the data, and the variance should also be kept small by minimizing the impact of data disturbance.

Bias and Variance Tradeoff

$$\text{bias} = \mathbb{E}_{\mathbf{x}}\left[\left(\bar{g}(\mathbf{x}) - f(\mathbf{x})\right)^2\right] \qquad \text{var} = \mathbb{E}_{\mathbf{x}}\left[\mathbb{E}_{\mathcal{D}}\left[\left(g^{(\mathcal{D})}(\mathbf{x}) - \bar{g}(\mathbf{x})\right)^2\right]\right]$$



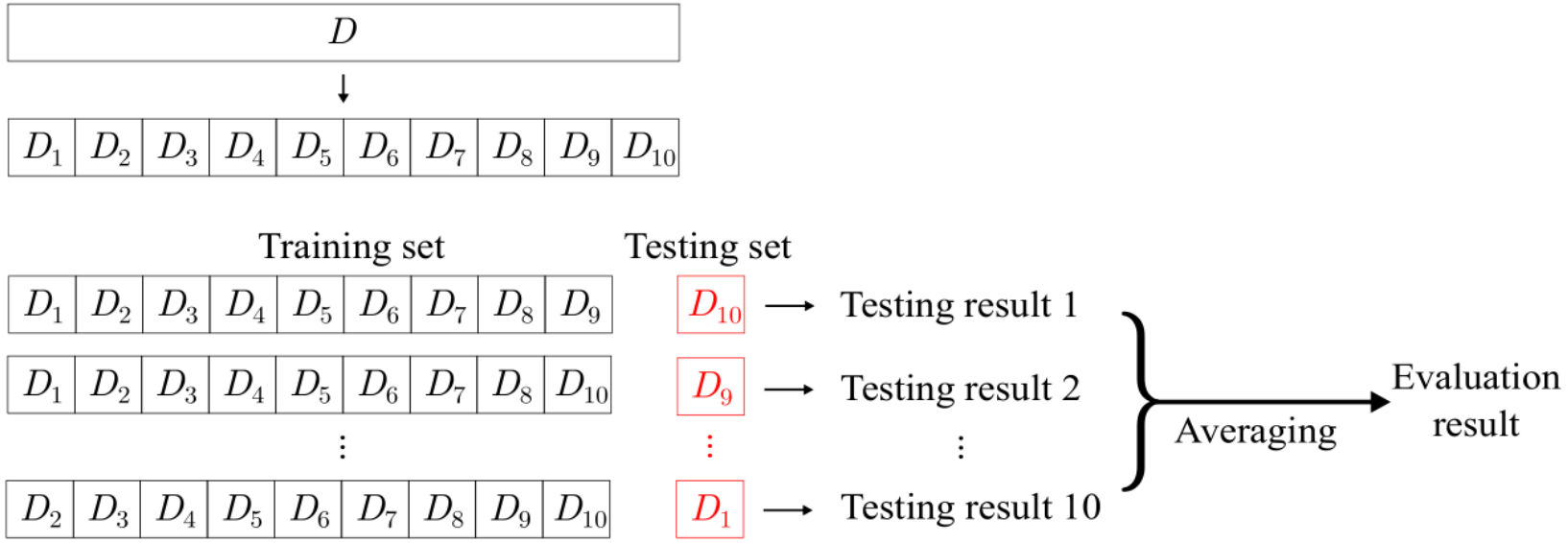


$\downarrow$ $\qquad$ $\mathcal{H} \uparrow$ $\qquad$ $\uparrow$

Cross-Validation

Cross-validation splits data set D into k disjoint subsets with similar sizes. In each trial of cross-validation, we use the union of $k-1$ subsets as the training set to train a model and then use the remaining subset as the testing set to evaluate the model.

We repeat this process k times and average over k trials to obtain the evaluation result. The most commonly used value of k is 10.



10-fold cross-validation

Lecture 2

Linear Regression

Linear Regression – loss function

- Minimize mean-squared error (MSE):

  Loss function: How much $\hat{y}$ differs from the true y

$$E_{(w,b)} = \sum_{i=1}^{m} (y_i - wx_i - b)^2$$

- Calculate the derivatives of $E_{(w,b)}$ with respect to w and b :

$$\frac{\partial E_{(w,b)}}{\partial w} = 2\left(w\sum_{i=1}^{m} x_i^2 - \sum_{i=1}^{m}(y_i - b)x_i\right)$$

$$\frac{\partial E_{(w,b)}}{\partial b} = 2\left(mb - \sum_{i=1}^{m}(y_i - wx_i)\right)$$

Linear Regression - Least Square Method

- We have the closed-form solutions

$$w = \frac{\sum_{i=1}^{m} y_i(x_i - \bar{x})}{\sum_{i=1}^{m} x_i^2 - \frac{1}{m}\left(\sum_{i=1}^{m} x_i\right)^2}$$

$$b = \frac{1}{m}\sum_{i=1}^{m}(y_i - wx_i)$$

where $\bar{x} = \frac{1}{m}\sum_{i=1}^{m} x_i$

Multivariate Linear Regression

- Rewrite $\boldsymbol{w}$and$b$ as $\hat{\boldsymbol{w}} = (\boldsymbol{w}; b)$, the data set is represented as

$$\mathbf{X} = \begin{pmatrix} x_{11} & x_{12} & \cdots & x_{1d} & 1 \\ x_{21} & x_{22} & \cdots & x_{2d} & 1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ x_{m1} & x_{m2} & \cdots & x_{md} & 1 \end{pmatrix} = \begin{pmatrix} \boldsymbol{x}_1^{\mathrm{T}} & 1 \\ \boldsymbol{x}_2^{\mathrm{T}} & 1 \\ \vdots & \vdots \\ \boldsymbol{x}_m^{\mathrm{T}} & 1 \end{pmatrix}$$

$$\boldsymbol{y} = (y_1; y_2; \ldots; y_m)$$

Multivariate Linear Regression - Least Square Method

- Least square method

$$\hat{\boldsymbol{w}}^* = \arg\min_{\hat{\boldsymbol{w}}} (\boldsymbol{y} - \mathbf{X}\hat{\boldsymbol{w}})^{\mathrm{T}} (\boldsymbol{y} - \mathbf{X}\hat{\boldsymbol{w}})$$

Let $E_{\hat{\boldsymbol{w}}} = (\boldsymbol{y} - \mathbf{X}\hat{\boldsymbol{w}})^{\mathrm{T}} (\boldsymbol{y} - \mathbf{X}\hat{\boldsymbol{w}})$ and find the derivative with respect to $\hat{\boldsymbol{w}}$

$$\frac{\partial E_{\hat{\boldsymbol{w}}}}{\partial \hat{\boldsymbol{w}}} = 2\mathbf{X}^{\mathrm{T}} (\mathbf{X}\hat{\boldsymbol{w}} - \boldsymbol{y})$$

The closed-form solution of $\hat{\boldsymbol{w}}$ can be obtained by making the equation equal to 0.

Multivariate Linear Regression - Least Square Method

- If $\mathbf{X}^{\mathrm{T}}\mathbf{X}$ is a full-rank matrix or a positive definite matrix, then

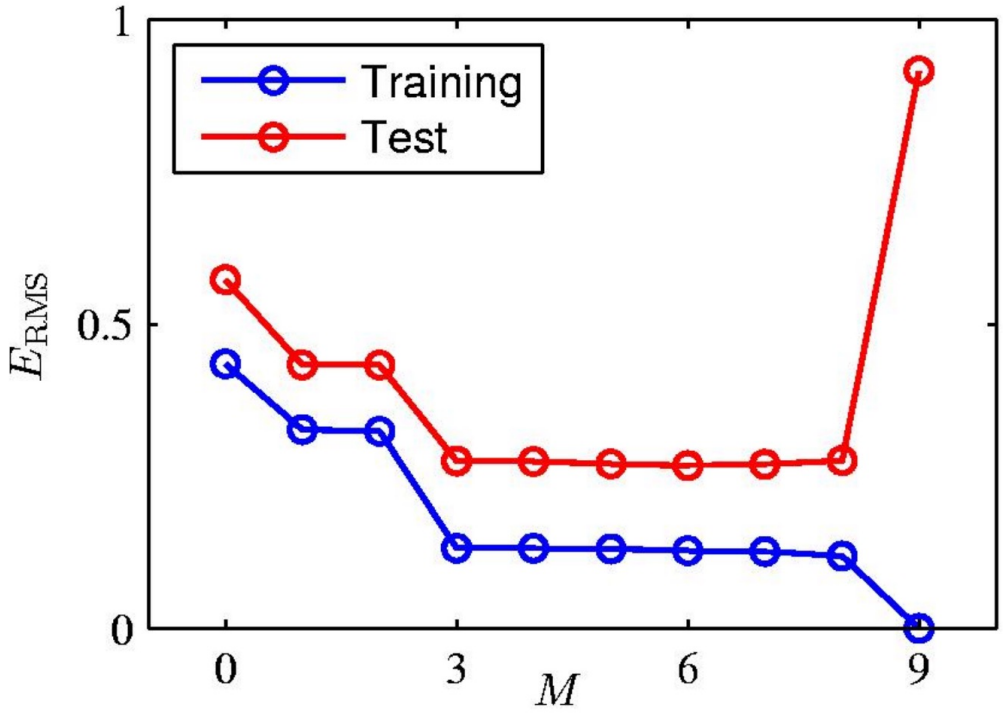
$$\hat{\boldsymbol{w}}^{*} = \left(\mathbf{X}^{\mathrm{T}}\mathbf{X}\right)^{-1}\mathbf{X}^{\mathrm{T}}\boldsymbol{y}$$

where $\left(\mathbf{X}^{\mathrm{T}}\mathbf{X}\right)^{-1}$ is the inverse of $\mathbf{X}^{\mathrm{T}}\mathbf{X}$, the learned multivariate linear regression model is

$$f\left(\hat{\boldsymbol{x}}_i\right) = \hat{\boldsymbol{x}}_i^{\mathrm{T}}\left(\mathbf{X}^{\mathrm{T}}\mathbf{X}\right)^{-1}\mathbf{X}^{\mathrm{T}}\boldsymbol{y}$$

- $\mathbf{X}^{\mathrm{T}}\mathbf{X}$ is often not full-rank
  - gradient descent (which is more broadly applicable)
  - pseudo-inverse

Overfitting



Root-Mean-Square (RMS) Error

Polynomial Coefficients

为了完美地穿过每一个训练数据点（包括噪声），模型需要产生非常“陡峭”和“曲折”的曲线。在数学上就体现为高阶项拥有非常大的权重，因为只有大的权重才能产生这种剧烈变化的函数形态。

	$M=0$	$M=1$	$M=3$	$M=9$
$w_0^\star$	0.19	0.82	0.31	0.35
$w_1^\star$		-1.27	7.99	232.37
$w_2^\star$			-25.43	-5321.83
$w_3^\star$			17.37	48568.31
$w_4^\star$				-231639.30
$w_5^\star$				640042.26
$w_6^\star$				-1061800.52
$w_7^\star$				1042400.18
$w_8^\star$				-557682.99
$w_9^\star$				125201.43

Regularization

Regularizer: a function that quantifies how much we prefer one hypothesis vs. another

Ridge regression and *Lasso*

- We can fit a model containing all features using a technique that constrains or regularizes the coefficient estimates, or equivalently, that **shrinks the coefficient estimates towards zero**.
- It may not be immediately obvious why such a constraint should improve the fit, but it turns out that shrinking the coefficient estimates can significantly reduce their variance.

Ridge Regression

- Recall that the least squares fitting procedure estimates $\beta_0, \beta_1, \ldots, \beta_p$ using the values that minimize

$$J(\boldsymbol{\beta}) = \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2$$

- In contrast, the ridge regression coefficient estimates are the values that minimize

$$J_{RR}(\boldsymbol{\beta}) = \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^{p} \beta_j^2$$

What's the solution?

$$\lambda ||\boldsymbol{\beta}||_2^2$$

where $\lambda \geq 0$ is a *tuning parameter*, to be determined separately.

Ridge Regression

- As with least squares, ridge regression seeks coefficient estimates that fit the data well.
- However, the second term, $\lambda \sum_j \beta_j^2$, called a *shrinkage penalty*, is small when $\beta_0, \beta_1, \ldots, \beta_p$ are close to zero, and so it has the effect of *shrinking* the estimates of β_j towards zero.
- The tuning parameter λ serves to control the relative impact of these two terms on the regression coefficient estimates.
- Selecting a good value for λ is critical; cross-validation is used for this.

Ridge Regression

Bayesian interpretation: MAP estimation with a **Gaussian prior** on the parameters

$$\boldsymbol{\beta}^{MAP} = \underset{\boldsymbol{\beta}}{\operatorname{argmax}} \sum_{i=1}^{m} \log p_{\boldsymbol{\beta}}(y^{(i)} \mid \boldsymbol{x}^{(i)}) + \log p(\boldsymbol{\beta})$$

$$= \underset{\boldsymbol{\beta}}{\operatorname{argmin}} J_{RR}(\boldsymbol{\beta})$$

where $p(\boldsymbol{\beta}) \sim \mathcal{N}(0, \frac{1}{\lambda})$

Ridge Regression

- Recall the gradient descent update:

$$\boldsymbol{\beta}^{t+1} \leftarrow \boldsymbol{\beta}^{t} - \alpha \frac{\partial \mathcal{J}}{\partial \boldsymbol{\beta}}$$

- The gradient descent update of the regularized cost has an interesting interpretation as weight decay:

$$\begin{aligned}
\boldsymbol{\beta}^{t+1} &\leftarrow \boldsymbol{\beta}^{t} - \alpha \left(\frac{\partial \mathcal{J}}{\partial \boldsymbol{\beta}} + \lambda \frac{\partial \mathcal{R}}{\partial \boldsymbol{\beta}} \right) \\
&= \boldsymbol{\beta}^{t} - \alpha \left(\frac{\partial \mathcal{J}}{\partial \boldsymbol{\beta}} + \lambda \boldsymbol{\beta}^{t} \right) \\
&= (1 - \alpha\lambda)\boldsymbol{\beta}^{t} - \alpha \frac{\partial \mathcal{J}}{\partial \boldsymbol{\beta}}
\end{aligned}$$

The Lasso

- Ridge regression does have one obvious disadvantage: ridge regression will include all features in the final model
- The Lasso is a relatively recent alternative to ridge regression that overcomes this disadvantage. The lasso coefficients minimize the quantity

What's the solution?

$$\sum_{i=1}^{n}\left(y_i-\beta_0-\sum_{j=1}^{p}\beta_j x_{ij}\right)^2+\lambda\sum_{j=1}^{p}|\beta_j| \quad \Leftrightarrow \quad \lambda||\boldsymbol{\beta}||_1$$

- In parlance, the lasso uses an ℓ_1 (pronounced "ell 1") penalty statistical instead of an ℓ_2 penalty.

Optimization for LASSO

- The L1 penalty is subdifferentiable (i.e. not differentiable at 0，高维空间中的非光滑函数)
- An array of optimization algorithms exist to handle this issue:
  - Coordinate Descent（坐标下降法，转化为一系列单变量子问题）
  - Block coordinate Descent (Tseng & Yun, 2009)
  - Sparse Reconstruction by Separable Approximation (SpaRSA) (Wright et al., 2009)
  - Fast Iterative Shrinkage Thresholding Algorithm (FISTA) (Beck & Teboulle, 2009)

The Lasso

- As with ridge regression, the lasso shrinks the coefficient estimates towards zero.
- However, in the case of the lasso, the ℓ_1 penalty has the effect of forcing some of the coefficient estimates to be exactly equal to zero when the tuning parameter λ is sufficiently large.
- Hence, much like best subset selection, the lasso performs *feature selection*.
- We say that the lasso yields *sparse* models — that is, models that involve only a subset of the variables.
- As in ridge regression, selecting a good value of λ for the lasso is critical; cross-validation is again the method of choice.

The Lasso

- Bayesian interpretation: MAP estimation with a **Laplace prior** on the parameters

$$\boldsymbol{\beta}^{MAP} = \underset{\boldsymbol{\beta}}{\operatorname{argmax}} \sum_{i=1}^{m} \log p_{\boldsymbol{\beta}}(y^{(i)} \mid \boldsymbol{x}^{(i)}) + \log p(\boldsymbol{\beta})$$

$$= \underset{\boldsymbol{\beta}}{\operatorname{argmin}} J_{RR}(\boldsymbol{\beta})$$

where $p(\boldsymbol{\beta}) \sim \text{Laplace}(0, f(\lambda))$

The Feature Selection Property of the Lasso

Why is it that the lasso, unlike ridge regression, results in coefficient estimates that are exactly equal to zero?

One can show that the lasso and ridge regression coefficient estimates solve the problems

$$\underset{\beta}{\text{minimize}} \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2 \quad \text{subject to} \quad \sum_{j=1}^{p} |\beta_j| \leq s$$

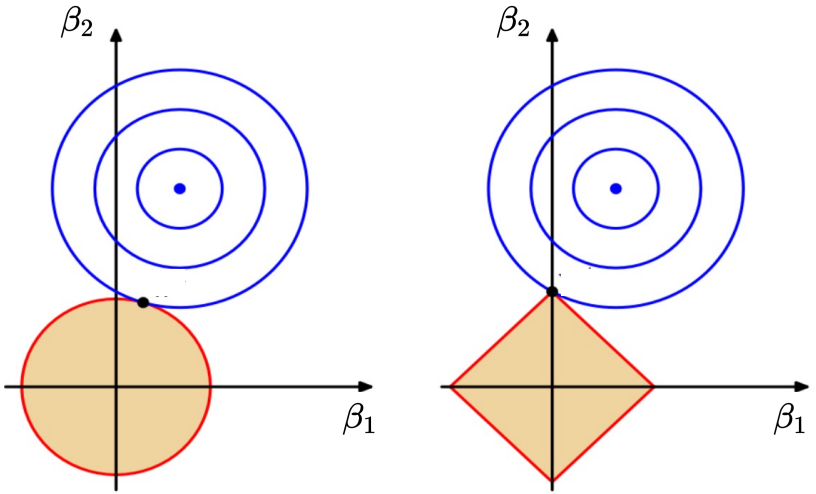
and

$$\underset{\beta}{\text{minimize}} \sum_{i=1}^{n} \left(y_i - \beta_0 - \sum_{j=1}^{p} \beta_j x_{ij} \right)^2 \quad \text{subject to} \quad \sum_{j=1}^{p} \beta_j^2 \leq s,$$

respectively.

Ridge Regression vs. Lasso

Why is it that the lasso, unlike ridge regression, results in coefficient estimates that are exactly equal to zero?

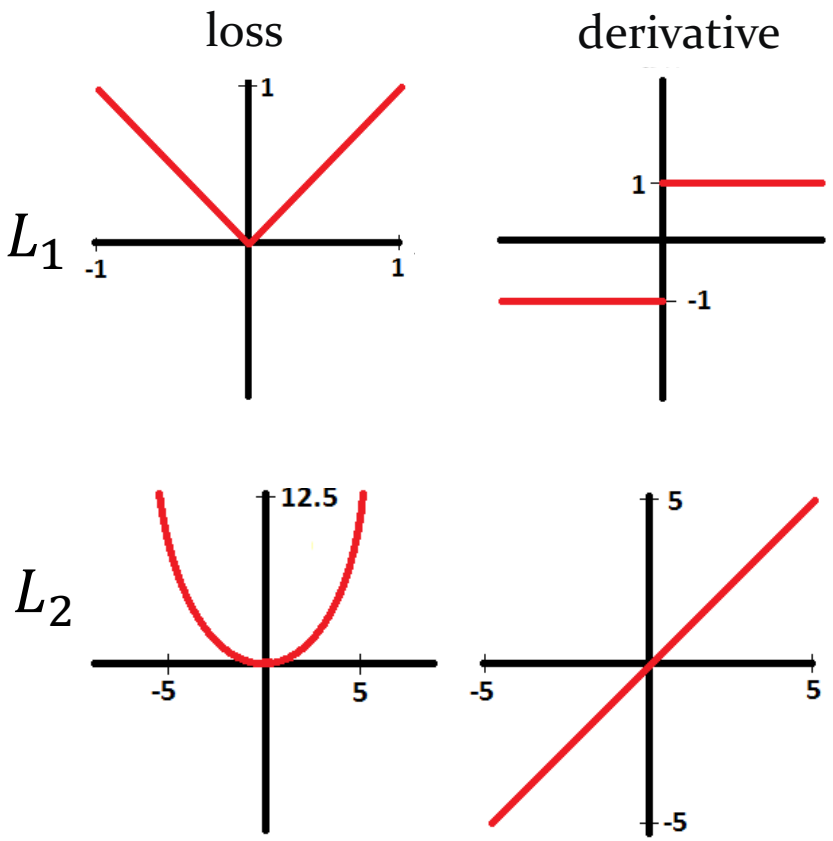


L2 regularization

$$\mathcal{R} = \sum_i \beta_i^2$$

L1 regularization

$$\mathcal{R} = \sum_i |\beta_i|$$



注：坐标下降法中，当参数估计值小于一个阈值（λ）时，优化算法会直接将其硬性设置为零

Regularization

- The linear model has distinct advantages in terms of its interpretability and often shows good predictive performance.

- ***Model Interpretability:*** By removing irrelevant features —that is, by setting the corresponding coefficient estimates to zero — we can obtain a model that is more easily interpreted. We will present some approaches for automatically performing *feature selection*.

Lecture 3

Logistic Regression

Binary Classification

- The predictions and the output labels

$$z = \boldsymbol{w}^{\mathrm{T}}\boldsymbol{x} + b \qquad y \in \{0, 1\}$$

- The real-valued predictions of the linear regression model need to be converted into **0/1**.

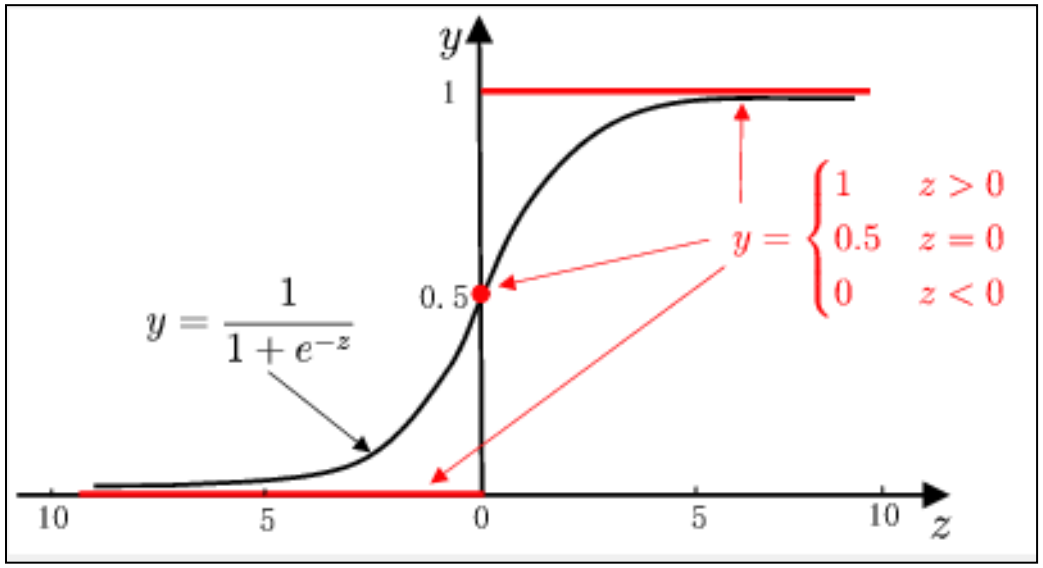
- Ideally, the unit-step function is desired

$$y = \begin{cases} 0, & z < 0; \\ 0.5, & z = 0; \\ 1, & z > 0, \end{cases}$$

  - which predicts positive for z greater than 0, negative for z smaller than 0, and an arbitrary output when z equals to 0.

Binary Classification

- Disadvantages of unit-step function
  - not continuous
- Logistic (sigmoid) function: a surrogate function to approximate the unit-step function
  - monotonic differentiable



Comparison between unit-step function and logistic function

$$\sigma(z) = \frac{1}{1+e^{-z}}$$

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1-\sigma(z))$$

Logistic Regression

Data: Inputs are continuous vectors of length d. Outputs are discrete labels.

$$\mathcal{D} = \left\{ \boldsymbol{x}^{(i)}, y^{(i)} \right\}_{i=1}^{m} \text{ where } \boldsymbol{x} \in \mathbb{R}^d \text{ and } y \in \{0, 1\}$$

Model: Logistic function applied to dot product of parameters with input vector.

$$p_{\boldsymbol{\theta}}(y = 1 \mid \mathbf{x}) = \frac{1}{1 + \exp\left(-\boldsymbol{\theta}^T \mathbf{x}\right)}$$

Learning: finds the parameters that minimize some objective function.

$$\boldsymbol{\theta}^* = \arg\min_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$$

Prediction: Output is the most probable class.

$$\hat{y} = \underset{y \in \{0,1\}}{\operatorname{argmax}}\, p_{\boldsymbol{\theta}}(y|\mathbf{x})$$

Log odds

- Apply logistic function

$$y = \frac{1}{1 + e^{-z}} \quad \text{transform into} \quad y = \frac{1}{1 + e^{-(\boldsymbol{w}^T \boldsymbol{x} + b)}}$$

- Log odds
  - the logarithm of the relative likelihood of a sample being a positive sample

$$\ln \frac{y}{1 - y} = \boldsymbol{w}^{\mathrm{T}} \boldsymbol{x} + \mathrm{b}$$

- Logistic regression has several nice properties
  - without requiring any prior assumptions on the data distribution
  - it predicts labels together with associated probabilities
  - it is solvable with numerical optimization methods.

Logistic regression - maximum likelihood

- Maximum likelihood
  - Given the training dataset $\mathcal{D} = \{(\boldsymbol{x}_i, y_i)\}_{i=1}^{m}$
  - Maximizing the probability of each sample being predicted as the ground-truth label
    - the log-likelihood to be maximized is:

$$\ell(\boldsymbol{w}, b) = \log \prod_{i=1}^{m} p(y_i \mid \boldsymbol{x}_i; \boldsymbol{w}, b)$$

    - assumption that the training examples are independent:

$$\ell(\boldsymbol{w}, b) = \sum_{i=1}^{m} \log p(y_i \mid \boldsymbol{x}_i; \boldsymbol{w}, b)$$

Logistic regression - maximum likelihood

- Log odds can be rewritten as

$$\ln \frac{p(y=1 \mid \boldsymbol{x})}{p(y=0 \mid \boldsymbol{x})} = \boldsymbol{w}^{\mathrm{T}}\boldsymbol{x} + b$$

and consequently,

$$p(y=1 \mid \boldsymbol{x}) = \frac{e^{\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}+b}}{1+e^{\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}+b}} = \mathrm{sigmoid}(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x} + b)$$

$$\begin{aligned} p(y=0 \mid \boldsymbol{x}) = \frac{1}{1+e^{\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x}+b}} &= 1 - \mathrm{sigmoid}(\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x} + b) \\ &= \mathrm{sigmoid}(-(\boldsymbol{w}^{T}\boldsymbol{x} + b)) \end{aligned}$$

Logistic regression - maximum likelihood

- Transform into minimize negative log-likelihood
  - Let $\boldsymbol{\beta} = (\boldsymbol{w}; b)$, $\hat{\boldsymbol{x}} = (\boldsymbol{x}; 1)$, $\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x} + b$ can be rewritten as $\boldsymbol{\beta}^{\mathrm{T}}\hat{\boldsymbol{x}}$
  - Let $p_1\left(\hat{\boldsymbol{x}}_i; \boldsymbol{\beta}\right) = p\left(y = 1 \mid \hat{\boldsymbol{x}}; \boldsymbol{\beta}\right)$

    $p_0\left(\hat{\boldsymbol{x}}_i; \boldsymbol{\beta}\right) = p\left(y = 0 \mid \hat{\boldsymbol{x}}; \boldsymbol{\beta}\right) = 1 - p_1\left(\hat{\boldsymbol{x}}_i; \boldsymbol{\beta}\right)$

  the likelihood term in can be rewritten as

$$p\left(y_i \mid \boldsymbol{x}_i; \boldsymbol{w}_i, b\right) = y_i p_1\left(\hat{\boldsymbol{x}}_i; \boldsymbol{\beta}\right) + \left(1 - y_i\right) p_0\left(\hat{\boldsymbol{x}}_i; \boldsymbol{\beta}\right)$$

  - maximizing log-likelihood is equivalent to minimizing

$$J(\boldsymbol{\beta}) = \sum_{i=1}^{m}\left(-y_i\boldsymbol{\beta}^{\mathrm{T}}\hat{\boldsymbol{x}}_i + \log\left(1 + e^{\beta^{\mathrm{T}}\hat{\boldsymbol{x}}_i}\right)\right)$$

Logistic regression - maximum likelihood

- Transform into minimize negative log-likelihood
  - Let $\boldsymbol{\beta} = (\boldsymbol{w}; b)$, $\hat{\boldsymbol{x}} = (\boldsymbol{x}; 1)$, $\boldsymbol{w}^{\mathrm{T}}\boldsymbol{x} + b$ can be rewritten as $\boldsymbol{\beta}^{\mathrm{T}}\hat{\boldsymbol{x}}$
  - Let $p_1\left(\hat{\boldsymbol{x}}_i; \boldsymbol{\beta}\right) = p\left(y = 1 \mid \hat{\boldsymbol{x}}; \boldsymbol{\beta}\right)$

    $$p_0\left(\hat{\boldsymbol{x}}_i; \boldsymbol{\beta}\right) = p\left(y = 0 \mid \hat{\boldsymbol{x}}; \boldsymbol{\beta}\right) = 1 - p_1\left(\hat{\boldsymbol{x}}_i; \boldsymbol{\beta}\right)$$

    the likelihood term in can be rewritten as

    $$p(y_i \mid \hat{\boldsymbol{x}}_i; \hat{\boldsymbol{w}}_i, b) = p_1(\hat{\boldsymbol{x}}_i; \boldsymbol{\beta})^{y_i} p_0(\hat{\boldsymbol{x}}_i; \boldsymbol{\beta})^{1-y_i}$$

  - maximizing log-likelihood is equivalent to minimizing

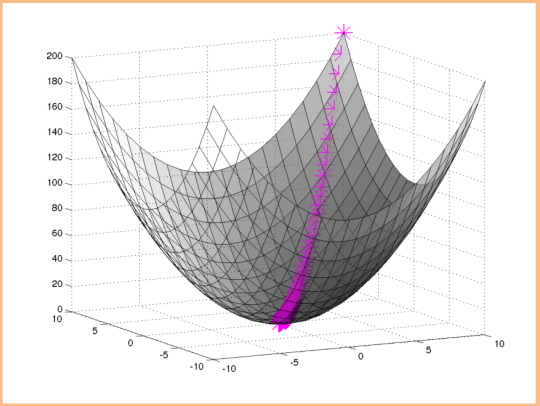
    $$J(\boldsymbol{\beta}) = \sum_{i=1}^{m} -[y_i \log p_1(\hat{\boldsymbol{x}}_i; \boldsymbol{\beta}) + (1 - y_i) \log p_0(\hat{\boldsymbol{x}}_i; \boldsymbol{\beta})]$$

    The Cross-Entropy loss!

Gradient Descent

Algorithm 1 Gradient Descent

1: **procedure** GD($\mathcal{D}$, $\boldsymbol{\theta}^{(0)}$)
2: $\quad\boldsymbol{\theta} \leftarrow \boldsymbol{\theta}^{(0)}$
3: $\quad$**while** not converged **do**
4: $\quad\quad\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$
5: $\quad$**return** $\boldsymbol{\theta}$



$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \begin{bmatrix} \frac{d}{d\theta_1} J(\boldsymbol{\theta}) \\ \frac{d}{d\theta_2} J(\boldsymbol{\theta}) \\ \vdots \\ \frac{d}{d\theta_N} J(\boldsymbol{\theta}) \end{bmatrix}$$

$$\boldsymbol{\theta}^{t+1} = \boldsymbol{\theta}^{t} - \eta \nabla J_{\boldsymbol{\theta}}(\boldsymbol{\theta}))$$

Gradient for Logistic Regression

- The cross-entropy loss function

$$J(\boldsymbol{\beta}) = \sum_{i=1}^{m} -[y_i \log p_1(\hat{\boldsymbol{x}}_i; \boldsymbol{\beta}) + (1 - y_i) \log p_0(\hat{\boldsymbol{x}}_i; \boldsymbol{\beta})]$$

- The gradient

$$\frac{\partial J(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}} = -\sum_{i=1}^{m} \hat{\boldsymbol{x}}_i (y_i - p_1(\hat{\boldsymbol{x}}_i; \boldsymbol{\beta}))$$

- Instead of using the sum notation, we can more efficiently compute the gradient in its matrix form

$$\frac{\partial J(\boldsymbol{\beta})}{\partial \boldsymbol{\beta}} = \mathbf{X}(\sigma(\mathbf{X}^T \boldsymbol{\beta}) - \mathbf{y})$$

$\mathbf{X} \in \mathbb{R}^{d \times m}$

σ : sigmoid

Lecture 5

Support Vector Machines

The Lagrange Method

Consider a general optimization problem (called as primal problem)

$$\begin{aligned} \min_x \quad & f(x) \\ \text{subject to} \quad & g_i(x) \geq 0, i = 1, \cdots, k \\ & h_j(x) = 0, j = 1, \cdots, m. \end{aligned}$$

We define its Lagrangian as

$$L(x, u, v) = f(x) - \sum_{i=1}^{k} \lambda_i g_i(x) + \sum_{j=1}^{m} u_j h(x)$$

Lagrangian multipliers $\lambda \in \mathbb{R}^k, u \in \mathbb{R}^m$.

The Dual Problem

A re-written Primal Problem :

$$\min_{x} \max_{\lambda \geq 0, u} L(x, \lambda, u)$$

The Dual Problem:

$$\max_{\lambda \geq 0, u} \min_{x} L(x, \lambda, u)$$

Although the primal problem is not required to be convex, the dual problem is always convex.

Theorem (weak duality):

$$d^* = \max_{\lambda \geq 0, u} \min_{x} L(x, \lambda, u) \leq \min_{x} \max_{\lambda \geq 0, u} L(x, \lambda, u) = p^*$$

Theorem (strong duality, *e.g.*, *Slater's condition*):
If the primal is a convex problem, and there exists at least one strictly feasible $\tilde{x}$, meaning that $\exists \tilde{x}, g_i(\tilde{x}) > 0, i = 1, \dots, k,\ h_j(\tilde{x}) = 0, j = 1, \dots, m.$

$$d^* = p^*$$

Karush–Kuhn–Tucker (KKT) conditions

Necessary conditions

If x^* and λ^*, u^* are the primal and dual solutions respectively with zero duality gap, we will show that x^*, λ^*, u^* satisfy the KKT conditions.

$$
\begin{aligned}
f(x^*) &= d(\lambda^*, u^*) \text{ by zero duality gap assumption} \\
&= \min_x f(x) - \sum_{i=1}^{k} \lambda_i^* g_i(x) + \sum_{j=1}^{m} u_j^* h_j(x), \text{ by definition} \\
&\leq f(x^*) - \sum_{i=1}^{k} \lambda_i^* g_i(x^*) + \sum_{j=1}^{m} u_j^* h_j(x^*) \\
&\leq f(x^*)
\end{aligned}
$$

stationarity

⇨ equality: x^* minimizes $L(x, \lambda^*, u^*)$

⇨ equality: $\lambda_i^* \, g_i(x^*) = 0$

complementary slackness

For convex problems with strong duality (e.g., when Slater's condition is satisfied), the KKT conditions are necessary and sufficient optimality conditions, i.e., x^* and (λ^*, u^*) are primal and dual optimal if and only if the KKT conditions hold.

The Primal Form of SVM

Maximum margin: finding the parameters $\boldsymbol{w}$ and b that maximize

$$\begin{aligned} \mathbf{w} \cdot \mathbf{x}_i + b &\geq 1, \quad \text{if } y_i = +1; \\ \mathbf{w} \cdot \mathbf{x}_i + b &\leq -1, \quad \text{if } y_i = -1 \end{aligned}$$

$$\begin{aligned} \arg\max_{\boldsymbol{w},b} \quad & \frac{2}{\|\boldsymbol{w}\|} \\ \text{s.t.} \quad & y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \geq 1, \quad i = 1, 2, \ldots, m. \end{aligned}$$

$$\begin{aligned} \arg\min_{\boldsymbol{w},b} \quad & \frac{1}{2}\|\boldsymbol{w}\|^2 \\ \text{s.t.} \quad & y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) \geq 1, \quad i = 1, 2, \ldots, m. \end{aligned}$$

This is an optimization problem with linear, inequality constraints.

Dual problem

- Lagrange multipliers
  - Step-1: introducing a Lagrange multiplier $\alpha_i \geq 0$, gives the Lagrange function

$$L(\boldsymbol{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\|\boldsymbol{w}\|^2 - \sum_{i=1}^{m} \alpha_i \left(y_i(\boldsymbol{w}^\top \boldsymbol{x}_i + b) - 1\right)$$

  - Step-2: Setting the partial derivatives of $L(\boldsymbol{w}, b, \boldsymbol{\alpha})$ with respect to $\boldsymbol{w}$ and b to 0 gives

$$\boldsymbol{w} = \sum_{i=1}^{m} \alpha_i y_i \boldsymbol{x}_i, \quad \sum_{i=1}^{m} \alpha_i y_i = 0.$$

  - Step-3: Substituting back

$$\begin{aligned} \min_{\boldsymbol{\alpha}} \quad & \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m} \alpha_i \alpha_j y_i y_j \boldsymbol{x}_i^\top \boldsymbol{x}_j - \sum_{i=1}^{m} \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^{m} \alpha_i y_i = 0, \ \alpha_i \geq 0, \ i = 1, 2, \ldots, m. \end{aligned}$$

Sparsity of the solution

- desired model:

$$f(\boldsymbol{x}) = \boldsymbol{w}^\top \boldsymbol{x} + b = \sum_{i=1}^{m} \alpha_i y_i \boldsymbol{x}_i^\top \boldsymbol{x} + b$$

- KKT conditions:

$$\boldsymbol{w} = \sum_{i=1}^{m} \alpha_i y_i \boldsymbol{x}_i, \quad \sum_{i=1}^{m} \alpha_i y_i = 0.$$ stationarity

$$\begin{cases} \alpha_i \geq 0, & \text{dual constraints} \\ y_i f(\boldsymbol{x}_i) \geq 1, & \text{primal constraints} \\ \alpha_i (y_i f(\boldsymbol{x}_i) - 1) = 0. & \text{complementary slackness} \end{cases}$$

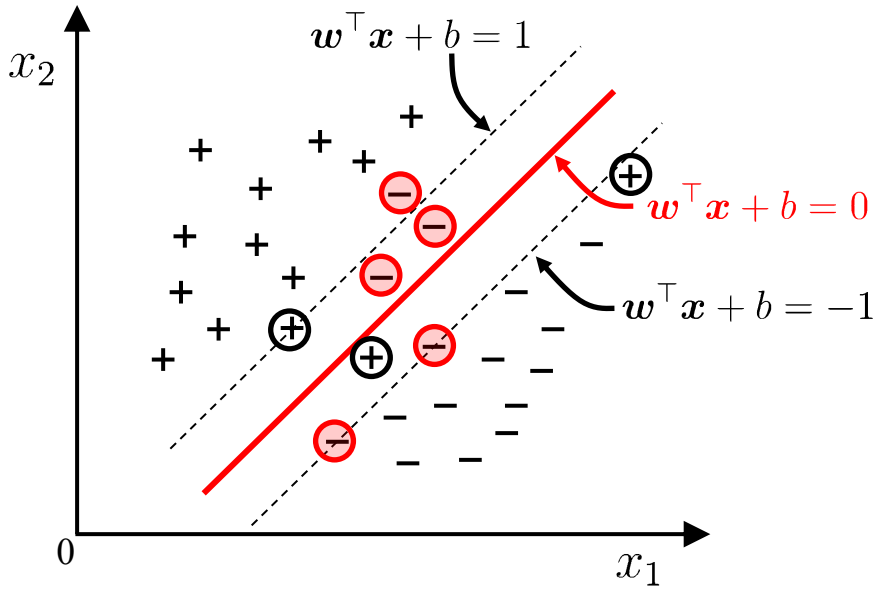
$y_i f(\boldsymbol{x}_i) > 1 \Rightarrow \alpha_i = 0$

Sparsity of the solution of SVM: once the training completed, most training samples are no longer needed since the final model only depends on the support vectors.

Key idea #2: the slack variables

-Q: It is often difficult to find an appropriate kernel function to make the training samples linearly separable in the feature space. Even if we do find such a kernel function, it is hard to tell if it is a result of overfitting.

-A: Allow a support vector machine to make mistakes on a few samples: *soft margin.*



Instances violating the constraint

ℓ_1 relaxation of the penalty term

The discrete nature of the penalty term on previous slide, $\sum_i 1_{\xi_i>0} = ||\vec{\xi}||_0$, makes the problem intractable.

A common strategy is to replace the ℓ_0 penalty with a ℓ_1 penalty: $\sum_i \xi_i = ||\vec{\xi}||_1$, resulting in the following full problem

$$\min_{\mathbf{w},b,\vec{\xi}} \frac{1}{2}\|\mathbf{w}\|_2^2 + C \cdot \sum_i \xi_i$$

$$\text{subject to} \quad y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i \text{ and } \xi_i \geq 0 \text{ for all } i.$$

Remarks:
(1) Also a quadratic program with linear ineq. constraints (just more variables): $y_i(\boldsymbol{w} \cdot \boldsymbol{x}_i + b) + \xi_i \geq 1$.

The Lagrange dual problem

The associated Lagrange function is

$$L(\mathbf{w}, b, \vec{\xi}, \vec{\lambda}, \vec{\mu}) = \frac{1}{2}\|\mathbf{w}\|_2^2 + C\sum_{i=1}^{n}\xi_i - \sum_{i=1}^{n}\lambda_i(y_i(\mathbf{w}\cdot\mathbf{x}_i + b) - 1 + \xi_i) - \sum_{i=1}^{n}\mu_i\xi_i$$

(stationary point) To find the dual problem we need to fix $\vec{\lambda}$, $\vec{\mu}$ and maximize over $\boldsymbol{w}, b, \vec{\xi}$:

$$\frac{\partial L}{\partial \mathbf{w}} = \mathbf{w} - \sum \lambda_i y_i \mathbf{x}_i = 0$$

$$\frac{\partial L}{\partial b} = \sum \lambda_i y_i = 0$$

$$\frac{\partial L}{\partial \xi_i} = C - \lambda_i - \mu_i = 0, \quad \forall i$$

The Lagrange dual problem

This yields the Lagrange dual function

$$L^*(\vec{\lambda}, \vec{\mu}) = \sum \lambda_i - \frac{1}{2} \sum \lambda_i \lambda_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j, \qquad \text{where}$$

$$\lambda_i \geq 0, \ \mu_i \geq 0, \ \lambda_i + \mu_i = C, \ \text{and} \ \sum \lambda_i y_i = 0.$$

The dual problem would be to maximize L^* over $\vec{\lambda}, \vec{\mu}$ subject to the constraints.

Since L^* is constant with respect to the μ_i, we can eliminate them to obtain a reduced dual problem:

$$\max_{\lambda_1, \ldots, \lambda_n} \sum \lambda_i - \frac{1}{2} \sum_{i,j} \lambda_i \lambda_j y_i y_j \mathbf{x}_i \cdot \mathbf{x}_j$$

$$\text{subject to} \quad \underbrace{0 \leq \lambda_i \leq C}_{\text{box constraints}} \ \text{and} \ \sum \lambda_i y_i = 0.$$

What changed?

What about the KKT conditions?

The KKT conditions are the following

$$\mathbf{w} = \sum \lambda_i y_i \mathbf{x}_i, \quad \sum \lambda_i y_i = 0, \quad \lambda_i + \mu_i = C$$

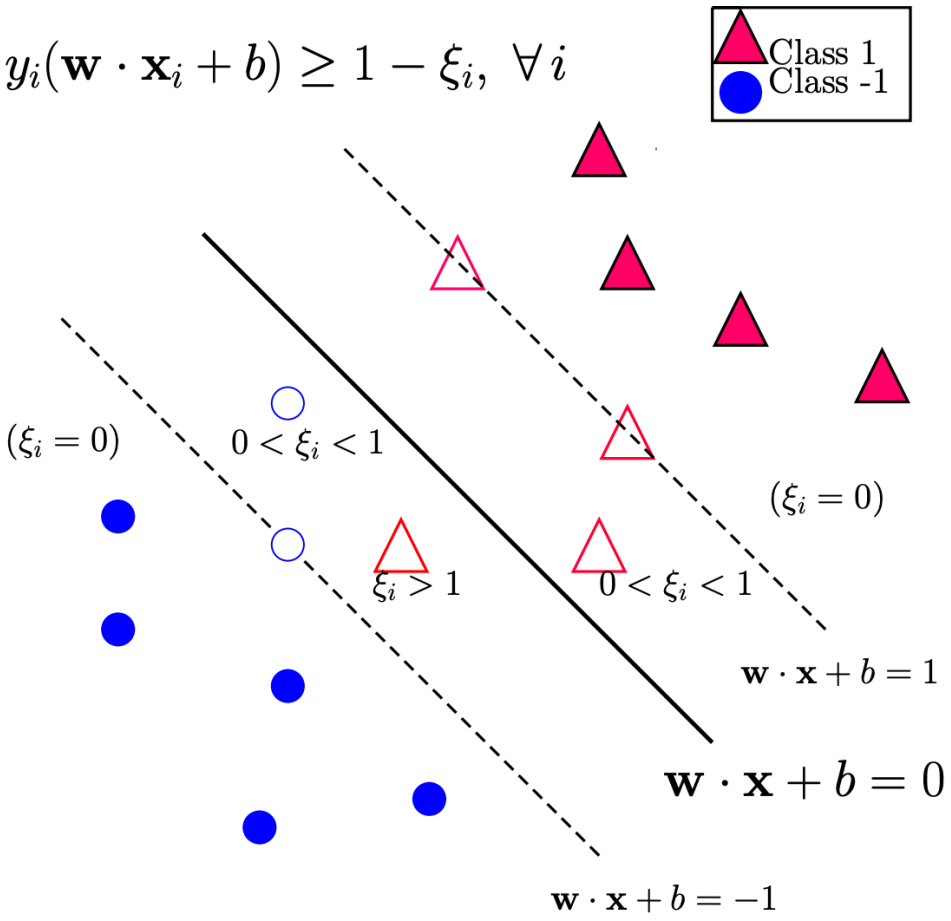
$$\lambda_i (y_i (\mathbf{w} \cdot \mathbf{x}_i + b) - 1 + \xi_i) = 0, \quad \mu_i \xi_i = 0$$

$$\lambda_i \geq 0, \quad \mu_i \geq 0$$

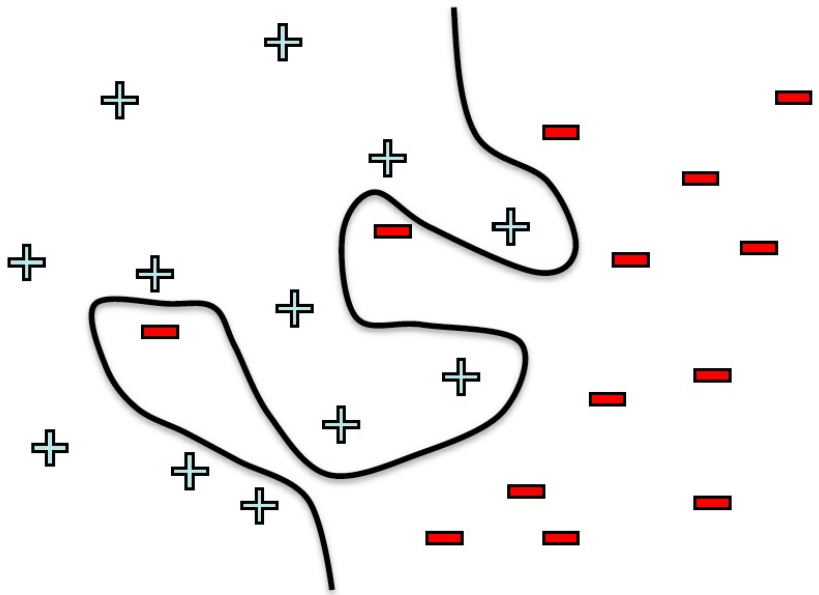
$$y_i (\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$$

We see that

- The optimal **w** has the same formula: $\boldsymbol{w} = \sum \lambda_i y_i \boldsymbol{x}_i$.
- Any point with $\lambda_i > 0$ and correspondingly $y_i(\boldsymbol{w} \cdot \boldsymbol{x} + b) = 1 - \xi_i$ is a support vector (not just those on the margin boundary $\boldsymbol{w} \cdot \boldsymbol{x} + b = \pm 1$).


$y_i(\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1 - \xi_i, \ \forall i$
Class 1
Class -1
$(\xi_i = 0)$
$0 < \xi_i < 1$
$(\xi_i = 0)$
$\xi_i > 1$
$0 < \xi_i < 1$
$\mathbf{w} \cdot \mathbf{x} + b = 1$
$\mathbf{w} \cdot \mathbf{x} + b = 0$
$\mathbf{w} \cdot \mathbf{x} + b = -1$

What if the data is not linearly separable?



Use features of features of features of features....

$$\phi(x) = \begin{pmatrix} x^{(1)} \\ \dots \\ x^{(n)} \\ x^{(1)}x^{(2)} \\ x^{(1)}x^{(3)} \\ \dots \\ e^{x^{(1)}} \\ \dots \end{pmatrix}$$

Kernel SVM

Let $\phi(\boldsymbol{x})$ denote the mapped feature vector of $\boldsymbol{x}$, the separating hyperplane $f(\boldsymbol{x}) = \boldsymbol{w}^\top \phi(\boldsymbol{x}) + b$ can be expressed as

Primal Problem

$$\begin{aligned} \min_{\boldsymbol{w},b} \quad & \frac{1}{2}\|\boldsymbol{w}\|^2 \\ \text{s.t.} \quad & y_i(\boldsymbol{w}^\top \phi(\boldsymbol{x}_i) + b) \geq 1, \quad i = 1, 2, \ldots, m. \end{aligned}$$

Dual Problem

$$\begin{aligned} \min_{\boldsymbol{\alpha}} \quad & \frac{1}{2}\sum_{i=1}^{m}\sum_{j=1}^{m}\alpha_i\alpha_j y_i y_j \phi(\boldsymbol{x}_i)^\top \phi(\boldsymbol{x}_j) - \sum_{i=1}^{m}\alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^{m}\alpha_i y_i = 0, \quad \alpha_i \geq 0, \quad i = 1, 2, \ldots, m. \end{aligned}$$

Prediction

$$f(\boldsymbol{x}) = \boldsymbol{w}^\top \phi(\boldsymbol{x}) + b = \sum_{i=1}^{m}\alpha_i y_i \phi(\boldsymbol{x}_i)^\top \phi(\boldsymbol{x}) + b$$

What are good kernel functions?

- Linear kernel
  - $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = \phi(\boldsymbol{x}_i)\phi(\boldsymbol{x}_j) = \boldsymbol{x}_i \cdot \boldsymbol{x}_j$
- Polynomial
  - $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = (\boldsymbol{x}_i \cdot \boldsymbol{x}_j + 1)^{\mathrm{n}}$
- Gaussian (also called Radial Basis Function, or RBF)
  - $K(\boldsymbol{x}_i, \boldsymbol{x}_j) = e^{\frac{\left\|x_i - x_j\right\|^2}{2\sigma^2}}$
- …

Quadratic kernel

$$
\begin{aligned}
k(\mathbf{x},\mathbf{z}) &= (\mathbf{x}^T\mathbf{z}+c)^2 = \left(\sum_{j=1}^{n} x^{(j)}z^{(j)} + c\right)\left(\sum_{\ell=1}^{n} x^{(\ell)}z^{(\ell)} + c\right) \\
&= \sum_{j=1}^{n}\sum_{\ell=1}^{n} x^{(j)}x^{(\ell)}z^{(j)}z^{(\ell)} + 2c\sum_{j=1}^{n} x^{(j)}z^{(j)} + c^2 \\
&= \sum_{j,\ell=1}^{n} (x^{(j)}x^{(\ell)})(z^{(j)}z^{(\ell)}) + \sum_{j=1}^{n} (\sqrt{2c}x^{(j)})(\sqrt{2c}z^{(j)}) + c^2,
\end{aligned}
$$

Feature mapping given by:

$$
\mathbf{\Phi}(\mathbf{x}) = [x^{(1)2}, x^{(1)}x^{(2)}, \ldots, x^{(3)2}, \sqrt{2c}x^{(1)}, \sqrt{2c}x^{(2)}, \sqrt{2c}x^{(3)}, c]
$$

Lecture 7

Backpropagation

Backpropagation Summary

1. **Forward pass:** for each training example, compute the outputs for all layers:

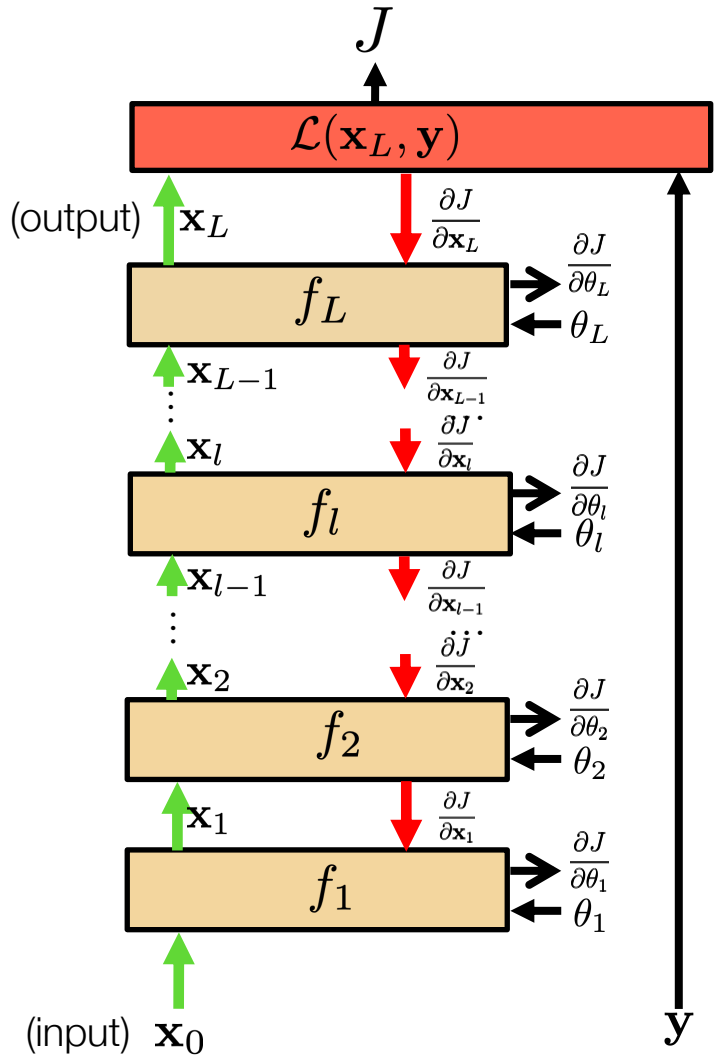
$$\mathbf{x}_l = f_l(\mathbf{x}_{l-1}, \theta_l)$$

2. **Backwards pass:** compute loss derivatives iteratively from top to bottom:

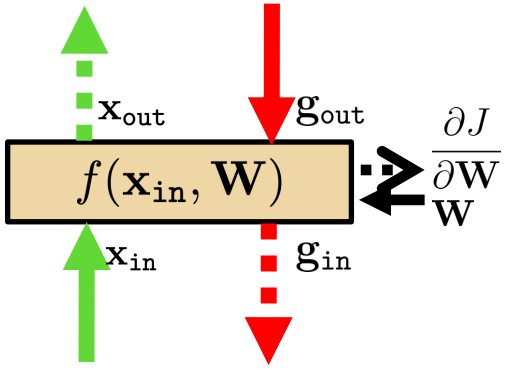
$$\frac{\partial J}{\partial \mathbf{x}_{l-1}} = \frac{\partial J}{\partial \mathbf{x}_l} \cdot \frac{\partial f_l}{\partial \mathbf{x}_{l-1}}$$

3. **Parameter update:** Compute gradients w.r.t. weights, and update weights:

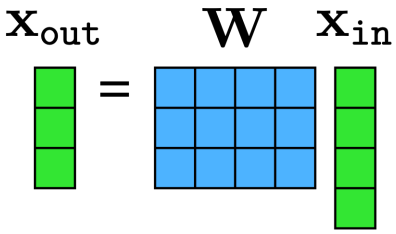
$$\frac{\partial J}{\partial \theta_l} = \frac{\partial J}{\partial \mathbf{x}_l} \cdot \frac{\partial f_l}{\partial \theta_l}$$



Linear layer



- Forward propagation: $\mathbf{x}_{\text{out}} = f(\mathbf{x}_{\text{in}}, \mathbf{W}) = \mathbf{W}\mathbf{x}_{\text{in}}$



With W being a matrix of size $|x_{out}| \times |x_{in}|$

- Backprop to input:

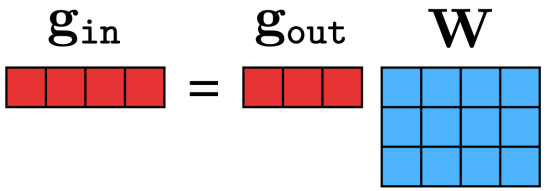
$$\mathbf{g}_{\text{in}} = \mathbf{g}_{\text{out}} \cdot \frac{\partial f(\mathbf{x}_{\text{in}}, \mathbf{W})}{\partial \mathbf{x}_{\text{in}}} = \mathbf{g}_{\text{out}} \cdot \frac{\partial \mathbf{x}_{\text{out}}}{\partial \mathbf{x}_{\text{in}}}$$

If we look at the i component of output x_{out}, with respect to the j component of the input, x_{in}:

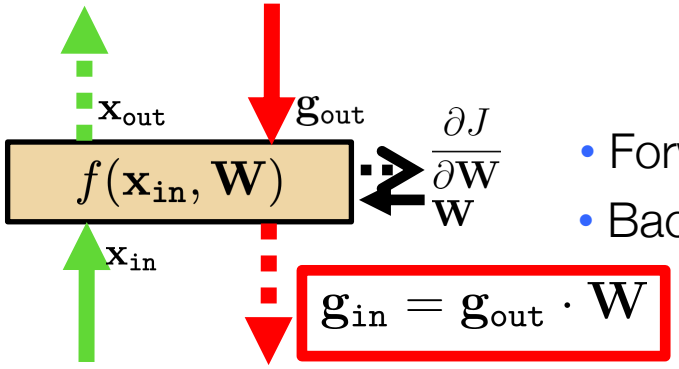
$$\frac{\partial \mathbf{x}_{\text{out}_i}}{\partial \mathbf{x}_{\text{in}_j}} = \mathbf{W}_{ij} \rightarrow \frac{\partial f(\mathbf{x}_{\text{in}}, \mathbf{W})}{\partial \mathbf{x}_{\text{in}}} = \mathbf{W}$$

Therefore:

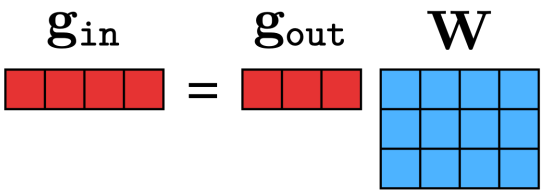
$$\mathbf{g}_{\text{in}} = \mathbf{g}_{\text{out}} \cdot \mathbf{W}$$



Linear layer

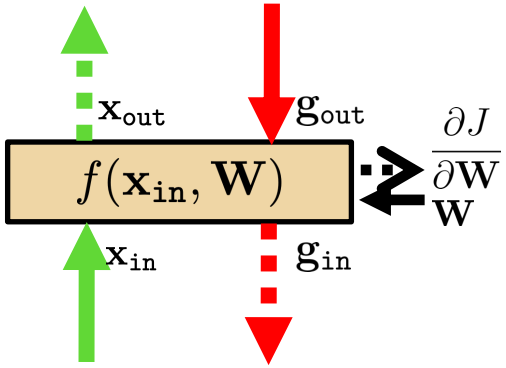


- Forward propagation: $\mathbf{x}_{\text{out}} = f(\mathbf{x}_{\text{in}}, \mathbf{W}) = \mathbf{W}\mathbf{x}_{\text{in}}$
- Backprop to input:



Now let's see how we use the set of outputs to compute the weights update equation (backprop to the weights).

Linear layer



- Forward propagation: $\mathbf{x}_{\text{out}} = f(\mathbf{x}_{\text{in}}, \mathbf{W}) = \mathbf{W}\mathbf{x}_{\text{in}}$
- Backprop to weights:

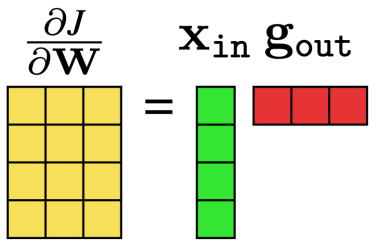
$$\frac{\partial J}{\partial \mathbf{W}} = \mathbf{g}_{\text{out}} \cdot \frac{\partial f(\mathbf{x}_{\text{in}}, \mathbf{W})}{\partial \mathbf{W}} = \mathbf{g}_{\text{out}} \cdot \frac{\partial \mathbf{x}_{\text{out}}}{\partial \mathbf{W}}$$

If we look at how the parameter W_{ij} changes the cost, only the i component of the output will change, therefore:

$$\frac{\partial J}{\partial \mathbf{W}_{ij}} = \frac{\partial J}{\partial \mathbf{x}_{\text{out}_i}} \cdot \frac{\partial \mathbf{x}_{\text{out}_i}}{\partial \mathbf{W}_{ij}} = \frac{\partial J}{\partial \mathbf{x}_{\text{out}_i}} \cdot \mathbf{x}_{\text{in}_j}$$

$$\frac{\partial \mathbf{x}_{\text{out}_i}}{\partial \mathbf{W}_{ij}} = \mathbf{x}_{\text{in}_j}$$

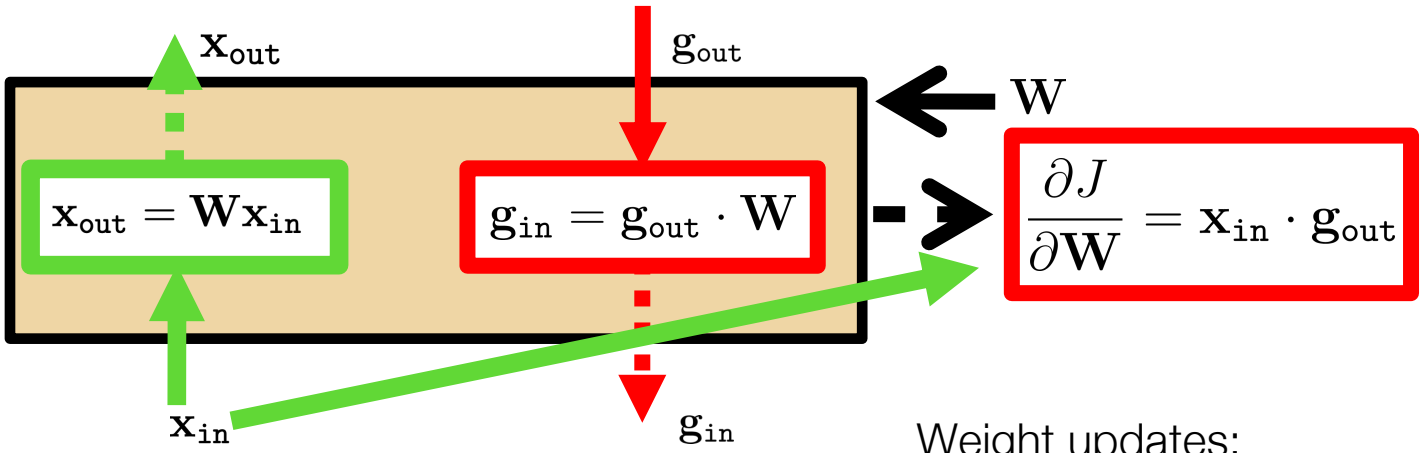
$$\frac{\partial J}{\partial \mathbf{W}} = \mathbf{x}_{\text{in}} \cdot \frac{\partial J}{\partial \mathbf{x}_{\text{out}}} = \mathbf{x}_{\text{in}} \cdot \mathbf{g}_{\text{out}}$$



And now we can update the weights:

$$\mathbf{W}^{k+1} \leftarrow \mathbf{W}^{k} + \eta \left(\frac{\partial J}{\partial \mathbf{W}} \right)^{T}$$

Linear layer



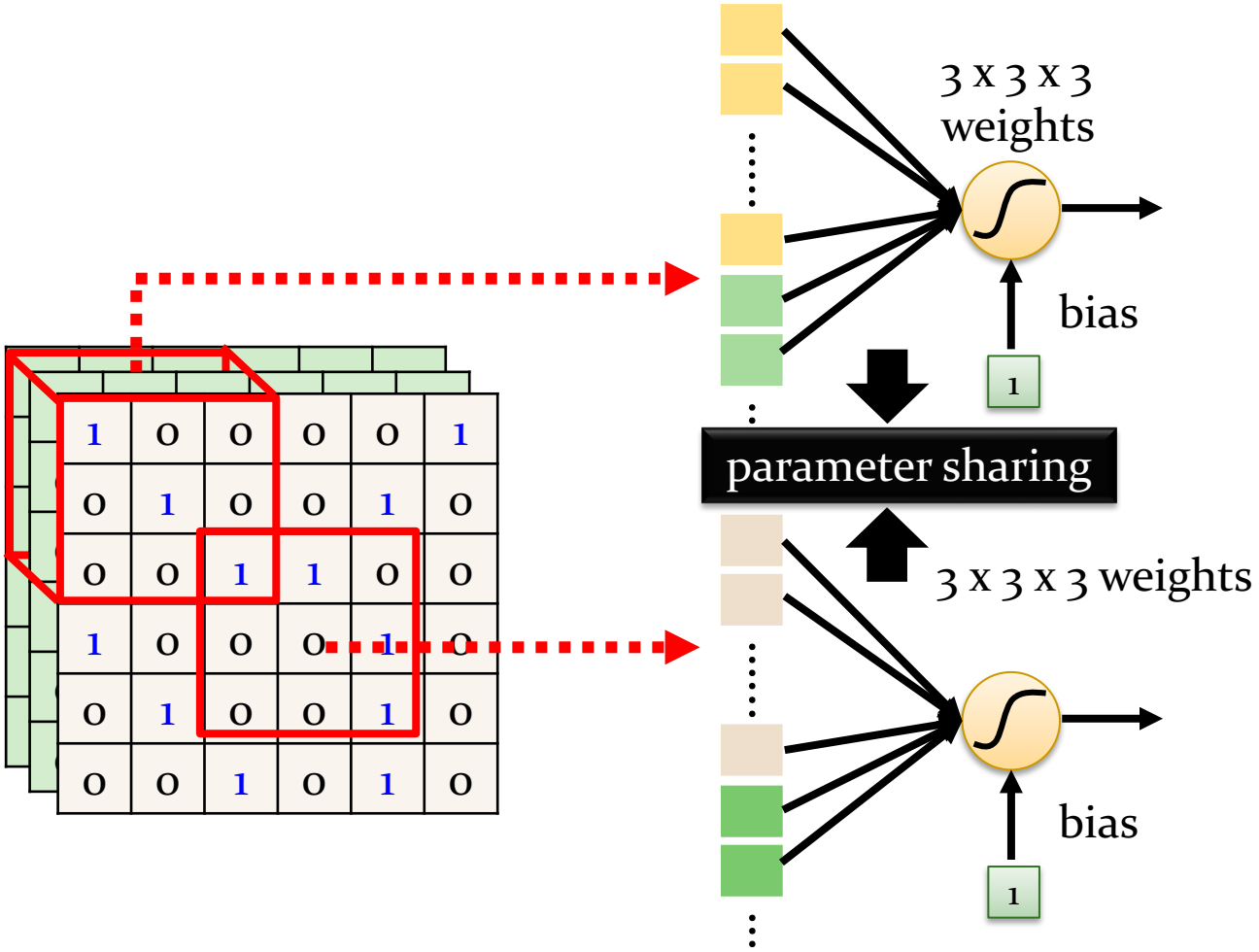
Weight updates:

$$\mathbf{W}^{k+1} \leftarrow \mathbf{W}^{k} + \eta \left(\frac{\partial J}{\partial \mathbf{W}} \right)^{T}$$

Lecture 8

Convolution Neural Network

Simplification 2



Pooling – Max Pooling

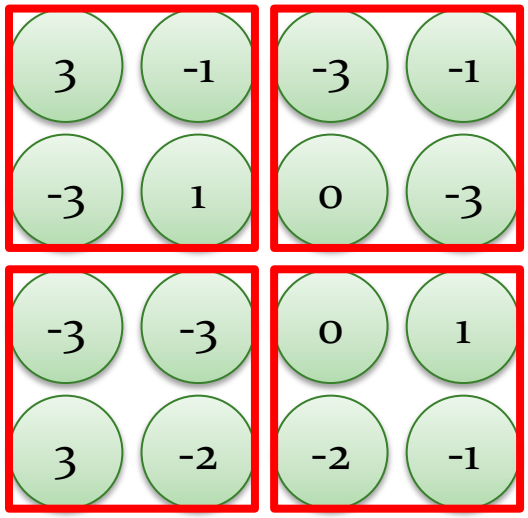
No learnable parameters!

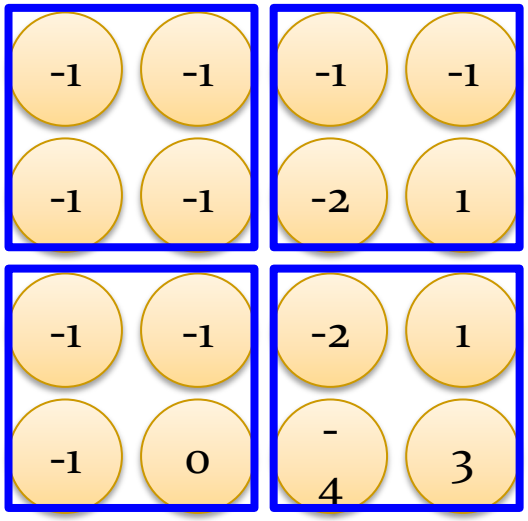
1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2





Chapter 10

Decision Tree

Basic Process

Algorithm 4.1 Decision Tree Learning.

Input: Training set $D = \{(x_1, y_1), (x_2, y_2), \ldots, (x_m, y_m)\}$;
Feature set $A = \{a_1, a_2, \ldots, a_d\}$.

Process: Function TreeGenerate(D, A)

```
1: Generate node i;
2: if All samples in D belong to the same class C then
3:     Mark node i as a class C leaf node; return
4: end if
5: if A = ∅ OR all samples in D take the same value on A then
6:     Mark node i as a leaf node, and its class label is the majority class in D; return
7: end if
8: Select the optimal splitting feature a_* from A;
9: for each value a_*^v in a_* do
10:    Generate a branch for node i; Let D_v be the subset of samples taking value a_*^v on a_*;
11:    if D_v is empty then
12:        Mark this child node as a leaf node, and label it with the majority class in D; return
13:    else
14:        Use TreeGenerate(D_v, A\{a_*}) as the child node.
15:    end if
16: end for
```

Output: A decision tree with root node i.

(1) All samples in the current node belong to the same class.

(2) The current feature set is empty, or all samples have the same feature values.

(3) There is no sample in the current node.

Split Selection: Information Gain

- Suppose that the discrete feature a has V possible values $\{a^1, a^2, ..., a^V\}$. Then, splitting the data set D by feature a produces V child nodes, where the vth child node D^v includes all samples in D taking the value a^v for feature a. Then, the *information gain* of splitting the data set D with feature a is calculated as

$$\text{Gain}(D, a) = \text{Ent}(D) - \sum_{v=1}^{V} \frac{|D^v|}{|D|} \text{Ent}(D^v)$$

is the importance of each node. The greater the number of samples, the greater the impact of the branch node.

- In general, the higher the information gain, the more purity improvement we can expect by splitting D with feature a.
- The decision tree algorithm ID3 [Quinlan, 1986] takes information gain as the guideline for selecting the splitting features.

Pruning

- Why pruning?
  - *Pruning* is the primary strategy of decision tree learning algorithms to deal with overfitting.
  - If there are too many branches, then the learner may be misled by the peculiarities of the training samples and incorrectly consider them as the underlying truth.
- General Pruning Strategies
  - *pre-pruning*
  - *post-pruning*
- How to evaluate generalization ability after pruning?
  - We can use the hold-out method to reserve part of the data as a validation set for performance evaluation.

Pruning: Pre-pruning

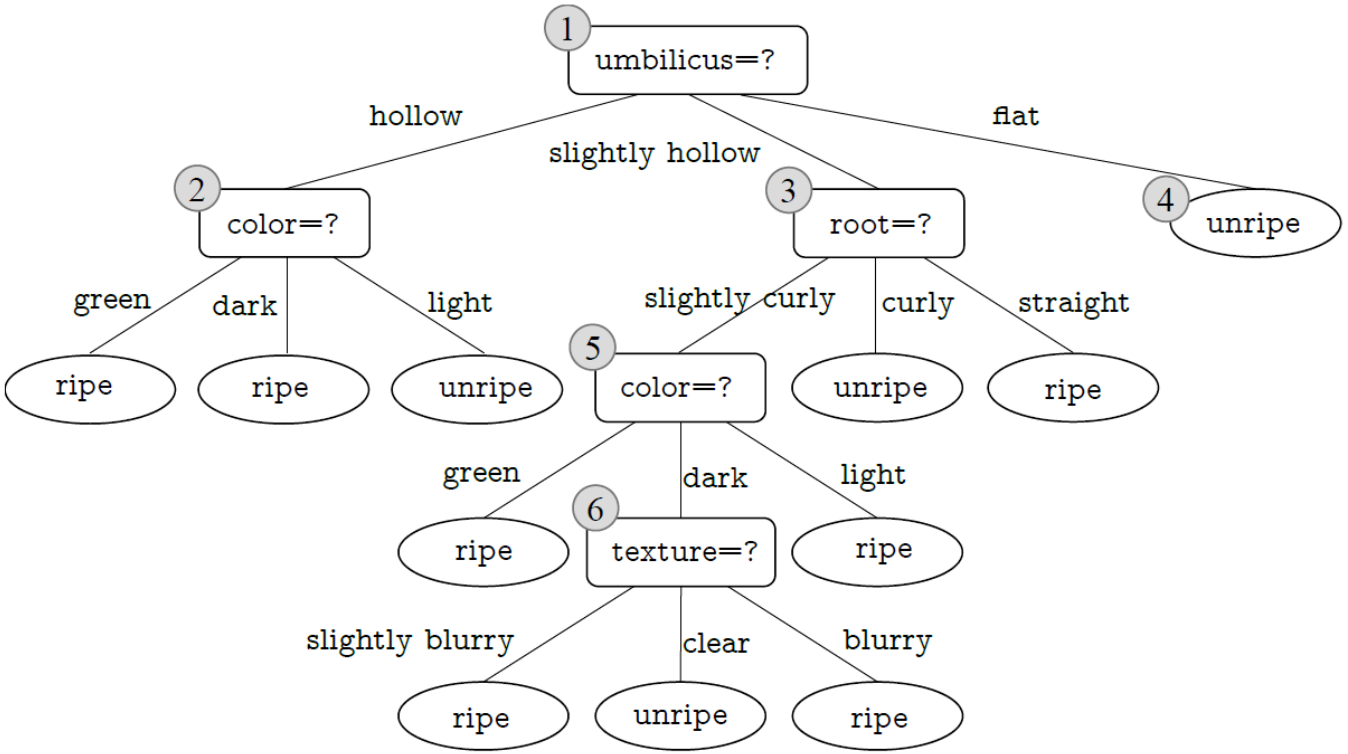
- Pre-pruning decides by comparing the generalization abilities before and after splitting.
  - If the validation accuracy decreases after pruning, the splitting is accepted.
  - Otherwise, the splitting is rejected.

- When no splitting is performed, this node is marked as a leaf node and its label is set to the majority class.

Pruning: Post-pruning

- Post-pruning allows a decision tree to grow into a complete tree. Then it takes a bottom-up strategy to examine every non-leaf node in the completely grown decision tree.

The validation accuracy of this decision tree is 42.9%



Chapter 11

Ensemble Learning

Bagging

- Bagging = Bootstrap AGGregatING

Algorithm 8.2 Bagging.

Input: Training set: $D = \{(\boldsymbol{x}_1, y_1), (\boldsymbol{x}_2, y_2), \ldots, (\boldsymbol{x}_m, y_m)\}$;
Base learning algorithm $\mathfrak{L}$;
Number of training rounds T.

Process:

1: **for** $t = 1, 2, \ldots, T$ **do**
2: $h_t = \mathfrak{L}(D, \mathcal{D}_{bs})$.
3: **end for**

Output: $H(\boldsymbol{x}) = \arg\max_{y \in \mathcal{Y}} \sum_{t=1}^{T} \mathbb{I}(h_t(\boldsymbol{x}) = y)$.

The bootstrap is one of the most important ideas in all of statistics!

Random Forests

- Random Forests = bagged decision trees, with one extra trick to decorrelate the predictions
  - When choosing each node of the decision tree, choose a random set of input features, and only consider splits on those features
- Random forests are probably the best black-box machine learning algorithm — they often work well with no tuning whatsoever.
  - one of the most widely used algorithms in Kaggle competitions

Chapter 12

Clustering

k-means Convergence

Objective

$$\min_{\mu}\min_{C} \sum_{i=1}^{k} \sum_{x \in C_i} |x - \mu_i|^2$$

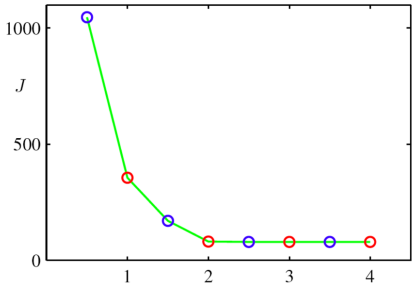
1. Fix μ, optimize C:

Step 1 of kmeans

$$\min_{C} \sum_{i=1}^{k} \sum_{x \in C_i} |x - \mu_i|^2 = \min_{c} \sum_{i}^{n} \left|x_i - \mu_{x_i}\right|^2$$

2. Fix C, optimize μ:

$$\min_{\mu} \sum_{i=1}^{k} \sum_{x \in C_i} |x - \mu_i|^2$$



– Take partial derivative of μ_i and set to zero, we have

$$\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$$

Step 2 of kmeans

Kmeans takes an alternating optimization approach, each step is guaranteed to decrease the objective – thus guaranteed to converge

[Slide from Alan Fern]

Hierarchical Clustering

- Hierarchical Clustering aims to create a tree-like clustering structure by dividing a data set at different layers. The hierarchy of clusters can be formed by taking either a *bottom-up* strategy (**Agglomerative，聚集**) or a *top-down* strategy (**Divisive，分裂**).
- **AGNES algorithm** (*bottom-up Hierarchical Clustering*)

starts by considering each sample in the data set as an initial cluster. Then, in each round, two nearest clusters are merged as a new cluster, and this process repeats until the number of clusters meets the pre-specified value.

We define the distances of given clusters C_i and C_j in different forms.

Hierarchical Clustering

Minimum distance (single-linkage , “单链接”):

$$d_{\min}(C_i, C_j) = \min_{\boldsymbol{x} \in C_i, \boldsymbol{z} \in C_j} \operatorname{dist}(\boldsymbol{x}, \boldsymbol{z})$$

Maximum distance (complete-linkage , “全链接”):

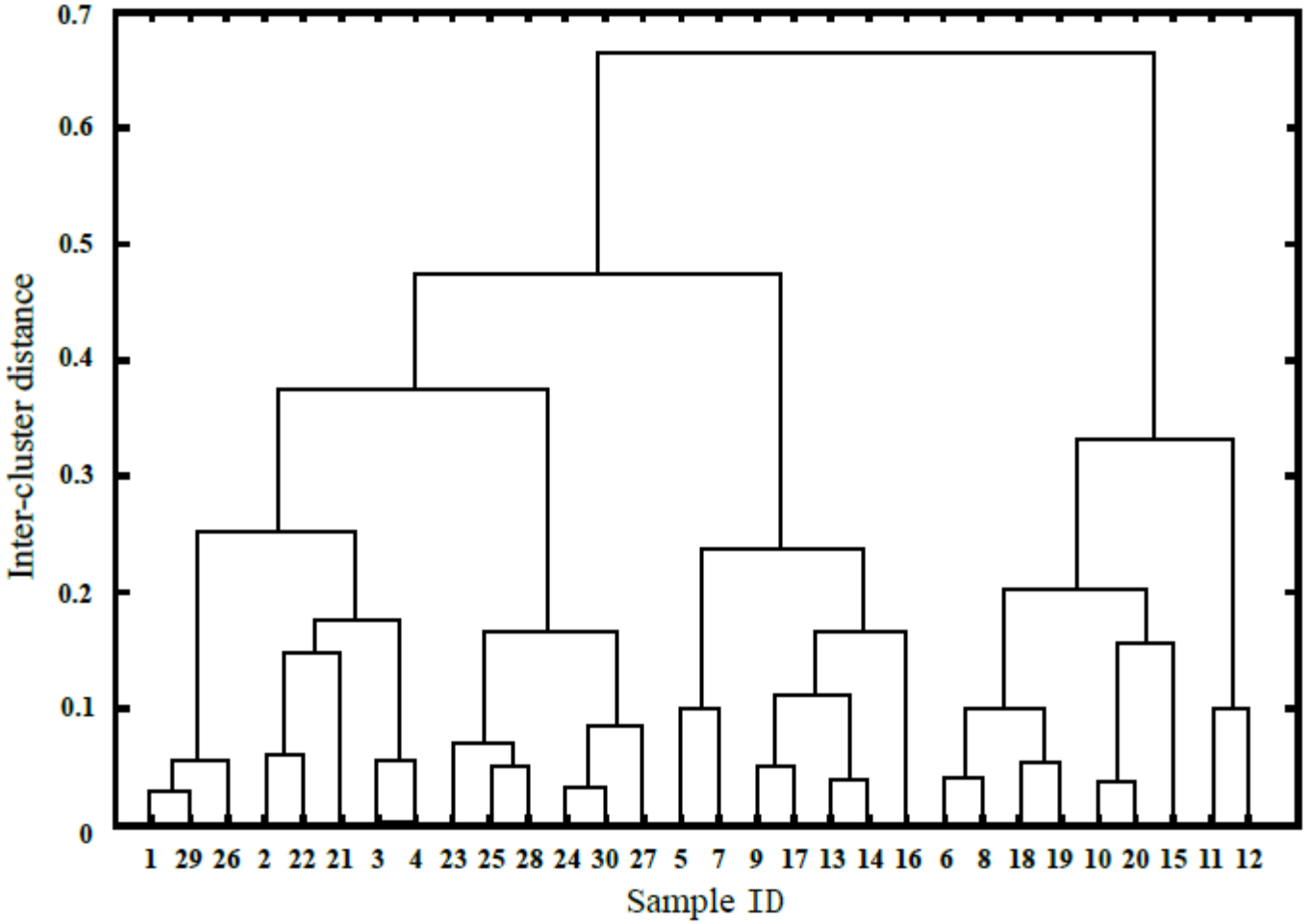
$$d_{\max}(C_i, C_j) = \max_{\boldsymbol{x} \in C_i, \boldsymbol{z} \in C_j} \operatorname{dist}(\boldsymbol{x}, \boldsymbol{z})$$

Average distance (average-linkage , “均链接”):

$$d_{\text{avg}}(C_i, C_j) = \frac{1}{|C_i||C_j|} \sum_{\boldsymbol{x} \in C_i} \sum_{\boldsymbol{z} \in C_j} \operatorname{dist}(\boldsymbol{x}, \boldsymbol{z})$$

Hierarchical Clustering – dendrogram

- The dendrogram (树状图) of AGNES :



Updating Distance Matrix

Let us assume that we have five samples (a, b, c, d, e) and the following matrix of pairwise distances between them:

	a	b	c	d	e
a	0	17	21	31	23
b	17	0	30	34	21
c	21	30	0	28	39
d	31	34	28	0	43
e	23	21	39	43	0

In this example, $D_1(a, b) = 17$ is the lowest value of D_1 so we cluster samples a and b.

Updating Distance Matrix

We then proceed to update the initial distance matrix D_1 into a new matrix D_2, reduced in size by one row and one column. Let's consider the **single-linkage** clustering:

$$
\begin{aligned}
D_2((a,b),c) &= \min(D_1(a,c), D_1(b,c)) = \min(21, 30) = 21 \\
D_2((a,b),d) &= \min(D_1(a,d), D_1(b,d)) = \min(31, 34) = 31 \\
D_2((a,b),e) &= \min(D_1(a,e), D_1(b,e)) = \min(23, 21) = 21
\end{aligned}
$$

	(a,b)	c	d	e
(a,b)	0	**21**	**31**	**21**
c	**21**	0	*28*	*39*
d	**31**	*28*	0	*43*
e	**21**	*39*	*43*	0

What if we adopt the complete-linkage clustering?